Copyright All rights reserved

1. 수

가상 세계를 구성하는 가장 작은 단위

2.1 수와 집합

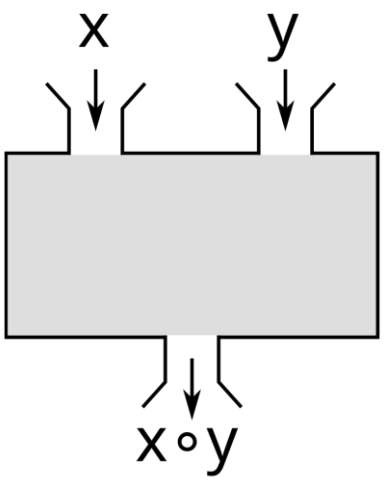
Heesung Oh
Copyright All rights reserved

- 수의 종류
  - 자연수
  - 정수
    - 양의정수
    - 0
    - 음의 정수
  - 실수
    - 유리수
    - 무리수
  - 복소수
    - real number + 복소수
  - 사원수
    - real number + 복소수 벡터

2.1.1 연산과 수의 구조

Heesung Oh
Copyright All rights reserved

- n항 연산(n-ary operation)
  - 어떤 집합 n개 원소의 순서쌍을 이 집합의 하나의 원소로 대응시키는 함수
- 이항 연산(Binary Operation)
  - 두 개의 피연산자를 입력으로 받아 하나의 결과를 생성하는 연산
- Closure Property(폐쇄성: 닫힘 성질)
  - 집합의 임의의 두 원소를 이항 연산 후, 결과가 다시 그 집합에 속하는 성질



2.1.1.1 이항 연산 성질

Heesung Oh
Copyright All rights reserved

- 교환법칙 (Commutative)
  - $a \otimes b = b \otimes a$
  - 연산 순서를 바꾸어도 결과가 같다
- 결합법칙 (Associative)
  - $(a \otimes b) \otimes c = a \otimes (b \otimes c)$
  - 연산을 묶는 순서를 바꾸어도 결과가 같다
- 항등원
  - $a \otimes e = a$
  - 연산 후 값이 변하지 않는 원소가 존재
- 역원
  - $a \otimes a^{-1} = e$
  - 연산 결과가 항등원이 되는 원소가 존재
- 분배 법칙 (Binary Operation)
  - 한 연산이 다른 연산에 대해 분배되는 성질

2.1.3 수의 표현

Heesung Oh
Copyright All rights reserved

- Number Line
  - 실수를 대응시켜 표현한 직선

- 크기와 방향
  - 값: 절대값
  - 방향: + , −
    - −2 → −|2|
  - 덧셈 연산
    - 뺄셈을 덧셈 연산
    - 1 – 5 → 1 + (−5)
  - 곱셈 연산
    - 2* (−2) → 2 * (−1) * |2| → (−1) * |2*2| → (−1) |4|

2.2 함수

Heesung Oh
Copyright All rights reserved

- 함수란
  - X{x} → Y{y} 로 대응할 때
    - X 의 모든 원소에 대한 대응 관계가 존재
    - x 는 하나의 y만 대응
- 정의역 (Domain)
  - 함수에 입력될 수 있는 값들의 집합
- 공역 (Codomain)
  - 함수의 결과가 속하도록 정해진 집합
- 치역 (Range)
  - 정의역의 값들이 실제로 대응되어 얻어지는 결과 값들의 집합

2.2.1 함수의 종류

Heesung Oh
Copyright All rights reserved

- 전사함수(Subjection: Onto)
  - 공역의 모든 요소가 정의역에 대응
  - 치열 == 공역
- 단사함수(Injection: One-to-One)
  - 정의역 과 공역의 요소가 1:1 대응
  - 치역 ⊂ 공역
- 전단사 함수(Bijection: One-to-One and Onto)
  - 정의역 과 공역의 요소가 1:1 대응
  - 치역 == 공역

2.2.2 합성함수

Heesung Oh
Copyright All rights reserved

- 함수의 합성 (Function Composition)
  - 함수의 연쇄
    - X{x} – f(x) → Y{y} – g(y) → Z{z}

- 합성 함수
  - 한 함수의 공역이 다른 함수의 정의역과 일치할 때, 두 함수를 차례로 적용하여 하나의 함수로 만든 것.
  - (f(x)) → g ○ f(x)
    - g circle f 또는 g after f

2.2.3 항등함수와 역함수

Heesung Oh
Copyright All rights reserved

- 항등함수 (Identify Function)
  - 정의역과 공역이 동일한 값 대응

- 역함수
  - 일대일 대응 f:X → Y 에서
    - $f^{-1}(y) = x$
    - 기호: $^{-1}$
    - 공역의 원소를 입력하면 원래 정의역의 원소를 반환하는 함수
  - 특성
    - $F \circ F^{-1}(x) = x$, $F \circ F^{-1}(y) = y$

- 합성함수의 역함수
  - $(g \circ f \circ h)^{-1} = h^{-1} \circ f^{-1} \circ g^{-1}$

2.2.4 좌표 평면

Heesung Oh
Copyright All rights reserved

- **곱집합 (Cartesian Product)**
  - 두 집합 A, B의 원소를 짝지어 만든 모든 순서쌍 (a,b) 집합
  - $A \times B = \{(a,b) \mid a \in A, b \in B\}$

- **2차원 좌표 평면**
  - 실수 집합 $\mathbb{R} \rightarrow \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}^2$
  - 2차원 좌표계
    - $\mathbb{R}^2$ 으로 하는 순서쌍의 집합
    - 곱집합 A → x축 , B → y 축

Copyright All rights reserved

3. 벡터

가상공간의 탄생

3.1 데카르트 좌표계

Heesung Oh
Copyright All rights reserved

- 좌표계
  - 공간에서 점의 위치를 수치로 표현하기 위한 기준 체계
  - 데카르트 좌표계(Cartesian coordinate system)
    - 서로 수직인 축을 기준으로 점의 위치를 좌표 값으로 표현
    - (x, y)
    - $\mathbb{R}^n$

- 좌표계 방향 (Handedness)
  - 오른손 좌표계 (Right-handed Coordinate System)
    - 오른손 엄지 X축 검지 Y축 중지 Z축
    - OpenGL, Vulkan, 3D MAX
  - 왼손 좌표계 (Left-handed Coordinate System)
    - 왼손 엄지 X축 검지 Y축 중지 Z축
    - DirectX, Unreal, Unity

3.2.1 스칼라와 벡터

Heesung Oh
Copyright All rights reserved

- **벡터**
  - 두개 이상의 실수의 곱집합
  - 간단히 실수로 구성된 튜플

- **벡터공간**
  - 벡터 덧셈, 스칼라 곱에 닫혀 있고, 일정한 대수적 규칙을 만족하는 집합
    - ※ 닫힘 성질(Closure): 연산의 결과가 피 연산자와 같은 집합에 속함

  - 벡터 덧셈 닫힘
    - V 공간에 임의의 벡터 v_1, v_2 를 더한 벡터도 V 공간 안에 존재

$$\overrightarrow{v_1} + \overrightarrow{v_2} \Rightarrow \vec{v}$$

  - 스칼라 곱 닫힘
    - V 공간에 임의의 벡터 v_1 에 스칼라를 더해도 V 공간 안에 존재

$$c\vec{v} \Rightarrow \overrightarrow{v'}$$

3.2.1 벡터의 종류

Heesung Oh
Copyright All rights reserved

- 벡터 종류
  - Zero Vector
    - 크기, 방향이 정의 되지 않는 특수한 벡터
  - 단위 벡터
    - 크기가 1인 벡터
  - 방향 벡터
    - 단위벡터 중에서 직선의 방향을 정의
  - 기저 벡터
    - 벡터 공간의 모든 공간을 선형 결합으로 표현할 수 있는 기본 벡터
  - 축 벡터
    - 회전축이나 회전 방향을 나타내는 벡터
  - 법선 벡터
    - 평면이나 직선에 수직인 벡터
  - 역 벡터
    - 크기는 같지만 방향이 정반대인 벡터
  - 성분 벡터
    - 하나의 벡터를 각 축의 방향으로 분해 했을 때 나타나는 벡터

3.2.2 벡터 공간의 연산

Heesung Oh
Copyright All rights reserved

- **벡터 덧셈 (벡터의 합)**

$$\overrightarrow{v_1} + \overrightarrow{v_2} = (x_1,\ y_1) + (x_2,\ y_2) = (x_1 + x_2,\ y_1 + y_2)$$

- **스칼라와 벡터의 곱셈 (스칼라 곱)**

$$c \times \vec{v} = (c \times x_1,\ \ c \times y_1)$$

$$C_i = A_i + B_i$$

$$C_i = cA_i$$

- **벡터 공간의 8가지 공리**

	공리	표현
벡터 합	결합법칙	(u + v) + w = u + (v + w)
	교환법칙	u + v = v + u
	항등원 존재	v + 0 = v
	역원 존재	v + (–v) = 0
스칼라 곱	곱셈 호환성	a(bv) = (ab)v
	항등원 존재	1v = v
	벡터 덧셈 분배법칙	a(u + v) = au + av
	스칼라 덧셈 분배법칙	(a + b)v = av + bv

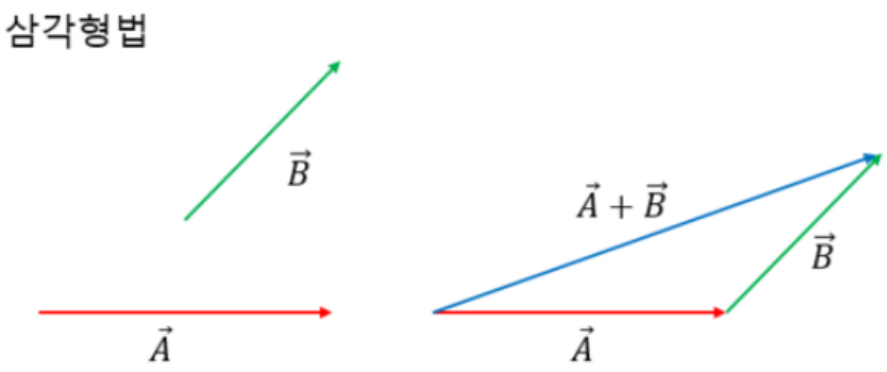
3.2.2 벡터 공간의 연산

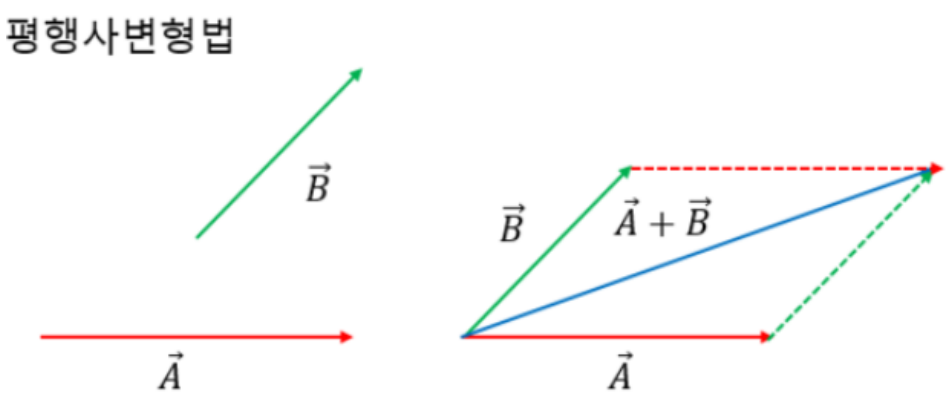
Heesung Oh
Copyright All rights reserved

- **벡터 덧셈 교환 법칙**

$$\begin{aligned}\overrightarrow{v_1}+\overrightarrow{v_2} &= (x_1, y_1)+(x_2, y_2)=(x_1+x_2,\ y_1+y_2)\\ &= (x_2+x_1,\ y_2+y_1)\\ &= (x_2+x_1,\ y_2+y_1)\\ &= \overrightarrow{v_2}+\overrightarrow{v_1}\end{aligned}$$

- **벡터의 합성**
  - 삼각형법, 평행사변형법
  - 간단히 벡터의 덧셈으로 계산





3.1 실습 – 헬로 벡터

Heesung Oh
Copyright All rights reserved

- 실습
  - git-fork: https://git-fork.com/
  - Project: https://github.com/onlybooks/gamemath.git
  - MSVC 2022: https://aka.ms/vs/17/release/vs_community.exe
  - cmake
    - https://cmake.org/download/ → cmake-4.2.3-windows-x86_64.msi
  - CMake-VS-17-2022.bat 수정
    - cmake -G "Visual Studio 18 2026" -DCMAKE_POLICY_VERSION_MINIMUM=3.5 ..₩

- class Vector2D
  - 벡터의 합, 차, 스칼라 곱을 operator overloading으로 구현
    - operator +(Vector2D)
    - operator -(Vector2D)
    - operator *(float)
    - operator *(float)

3.2.3 벡터의 크기와 이동

Heesung Oh
Copyright All rights reserved

- **피타고라스 정리 사용**

$$c^2 = a^2 + b^2$$

- **벡터의 크기**

$$||\vec{v}|| = |\vec{v}| = \left(\sum v_i^2\right)^{\frac{1}{2}} = (\vec{v} \cdot \vec{v})^{\frac{1}{2}}$$

- **단위 벡터**
  - **크기: 1**

$$\hat{v} = \frac{\vec{v}}{|\vec{v}|}$$

Heesung Oh
Copyright All rights reserved

- **내적**
  - **동일 차원 내 두 벡터의 크기와 사이각(θ)의 코사인($\cos\theta$) 값의 곱**

$$\vec{A}\cdot\vec{B} = \|\vec{A}\|\|\vec{B}\|\cos\theta \quad = \|\vec{A}\|\cos\theta\,\|\vec{B}\|$$

$$\vec{A}\cdot\vec{B} = \sum A_k B_k \qquad = A_x B_x + A_y B_y + A_z B_z$$

$$\langle A|\cdot|B\rangle = \langle A|B\rangle$$

Heesung Oh
Copyright All rights reserved

두 벡터의 각도

$$\vec{C} = \vec{A} - \vec{B}$$

$$\begin{aligned}|\vec{C}|^2 &= \vec{C}\cdot\vec{C} \\ &= (\vec{A}-\vec{B})\cdot(\vec{A}-\vec{B}) \\ &= \vec{A}\cdot\vec{A} - \vec{A}\cdot\vec{B} - B\cdot\vec{A} + \vec{B}\cdot\vec{B} \\ &= \vec{A}\cdot\vec{A} + \vec{B}\cdot\vec{B} - 2\vec{A}\cdot\vec{B}\end{aligned}$$

$$\therefore |\vec{C}|^2 = |\vec{A}|^2 + |\vec{B}|^2 - 2\vec{A}\cdot\vec{B}$$

$$2\vec{A}\cdot\vec{B} = |\vec{A}|^2 + |\vec{B}|^2 - |\vec{C}|^2$$

$$\vec{A}\cdot\vec{B} = \left(\frac{|\vec{A}|^2 + |\vec{B}|^2 - |\vec{C}|^2}{2}\right)$$

$$\therefore |\vec{A}||\vec{B}|\cos\theta = \left(\frac{|\vec{A}|^2+|\vec{B}|^2-|\vec{C}|^2}{2}\right)$$

$$\cos\theta = \left(\frac{|\vec{A}|^2 + |\vec{B}|^2 - |\vec{C}|^2}{2|\vec{A}||\vec{B}|}\right)$$

$$\therefore \theta = cos^{-1}\left(\frac{|\vec{A}|^2 + |\vec{B}|^2 - |\vec{A}-\vec{B}|^2}{2|\vec{A}||\vec{B}|}\right)$$

$$\therefore \theta = cos^{-1}\left(\frac{|\vec{A}|^2 + |\vec{B}|^2 - |\vec{A}-\vec{B}|^2}{2\sqrt{|\vec{A}|^2|\vec{B}|^2}}\right)$$

Heesung Oh
Copyright All rights reserved

외적 (cross product)

- 두 벡터에 모두 수직인 새로운 벡터를 만드는 연산

$$\vec{C} = \vec{A} \times \vec{B}$$

- 크기: 두 벡터가 가지는 평형사변형 넓이

$$\left|\vec{A} \times \vec{B}\right| = \left|\vec{A}\right| \cdot \left|\vec{B}\right| \sin\theta$$

- 방향: 오른손 법칙(Right-Hand Rule)을 적용해서 두 벡터에 수직

- 행렬식으로 간단히 표현

$$\vec{C} = \vec{A} \times \vec{B} = \begin{vmatrix} \hat{x} & \hat{y} & \hat{z} \\ A_x & A_y & A_z \\ B_x & B_y & B_z \end{vmatrix}$$

$$\vec{A} \times \vec{B} = \left(A_y B_z - A_z B_y,\ A_z B_x - A_x B_z,\ A_x B_y - A_y B_x\right)$$

3.2 실습: 벡터 원 그리기

Heesung Oh
Copyright All rights reserved

- Project tree/3-2
- 벡터 크기

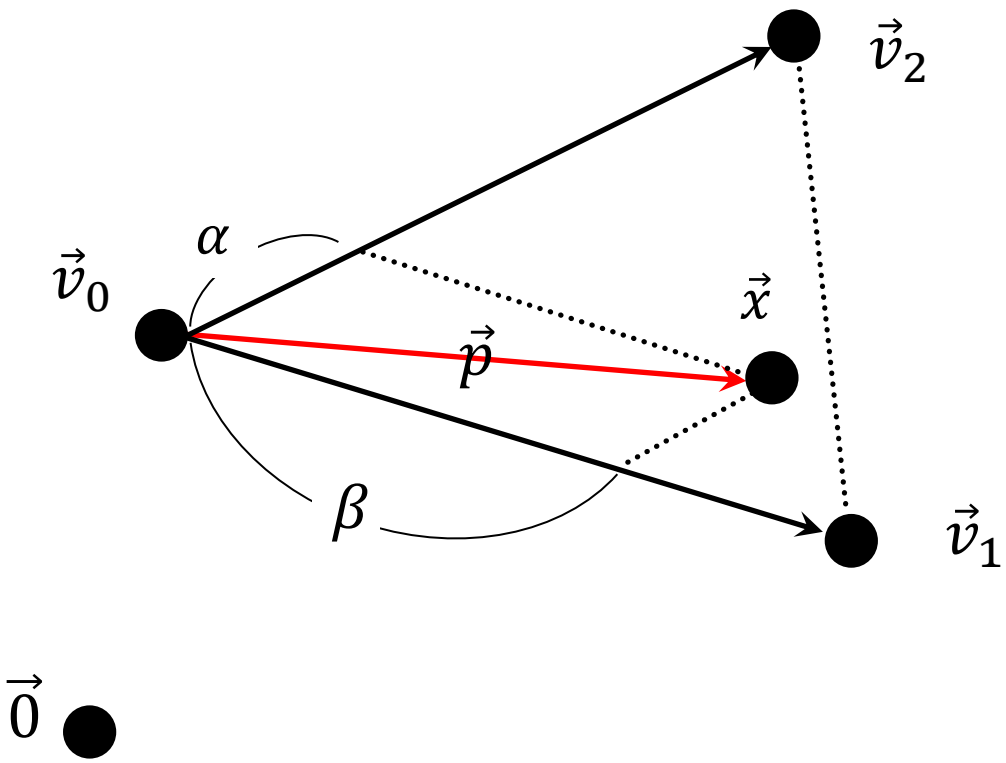
```
FORCEINLINE float Vector2::Size() const
{
        return sqrtf(SizeSquared());
}

FORCEINLINE constexpr float Vector2::SizeSquared() const
{
        return X * X + Y * Y;
}
```

- 반경 안 원소 모집

```
Vector2 pointToTest = Vector2(x, y);
float squaredLength = pointToTest.SizeSquared();
if (squaredLength <= radius * radius)
{
        circles.push_back(Vector2(x, y));
}
```

Heesung Oh
Copyright All rights reserved

- 삼각형 세 점 V0, V1, V2 와 X 벡터를 분해



Heesung Oh
Copyright All rights reserved

벡터 분해

$$\vec{A} = (\vec{v}_1 - \vec{v}_0) \quad \vec{B} = (\vec{v}_2 - \vec{v}_0) \quad \vec{p} = \vec{X} - \vec{B}$$

$$\vec{p} = \alpha \vec{A} + \beta \vec{B}$$

$$\vec{A} \cdot \vec{p} = \alpha \vec{A} \cdot \vec{A} + \beta \vec{A} \cdot \vec{B}$$

$$\vec{B} \cdot \vec{p} = \alpha \vec{A} \cdot \vec{B} + \beta \vec{B} \cdot \vec{B}$$

$$\begin{pmatrix} \vec{A} \cdot \vec{A} & \vec{A} \cdot \vec{B} \\ \vec{A} \cdot \vec{B} & \vec{B} \cdot \vec{B} \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} \vec{A} \cdot \vec{p} \\ \vec{B} \cdot \vec{p} \end{pmatrix}$$

$$\alpha = \frac{\begin{vmatrix} \vec{A} \cdot \vec{p} & \vec{A} \cdot \vec{B} \\ \vec{B} \cdot \vec{p} & \vec{B} \cdot \vec{B} \end{vmatrix}}{\begin{vmatrix} \vec{A} \cdot \vec{A} & \vec{A} \cdot \vec{B} \\ \vec{A} \cdot \vec{B} & \vec{B} \cdot \vec{B} \end{vmatrix}} \qquad \beta = \frac{\begin{vmatrix} \vec{A} \cdot \vec{A} & \vec{A} \cdot \vec{p} \\ \vec{A} \cdot \vec{B} & \vec{B} \cdot \vec{p} \end{vmatrix}}{\begin{vmatrix} \vec{A} \cdot \vec{A} & \vec{A} \cdot \vec{B} \\ \vec{A} \cdot \vec{B} & \vec{B} \cdot \vec{B} \end{vmatrix}}$$

$$\vec{x} = \overrightarrow{v_0} + \alpha (\overrightarrow{v_1} - \overrightarrow{v_0}) + \beta (\overrightarrow{v_2} - \overrightarrow{v_0})$$

3.3 벡터의 결합과 생성

Heesung Oh
Copyright All rights reserved

- 기저 벡터
  - 벡터 공간을 구성하는 기본 벡터 집합
  - 벡터: 기저 벡터의 선형 결합으로 표현 가능

- 2차원 공간 기저 벡터
  - $\hat{e}_1$ = (1, 0), $\hat{e}_2$ = (0, 1)
  - $\vec{v} = x\hat{e}_1 + y\hat{e}_2$

- 3차원 공간 기저 벡터
  - $\hat{e}_1$ = (1, 0, 0), $\hat{e}_2$ = (0, 1, 0), $\hat{e}_3$ = (0, 0, 1)
  - $\vec{v} = x\hat{e}_1 + y\hat{e}_2 + z\hat{e}_3$

$$\vec{v} = e_x\hat{e}_x$$

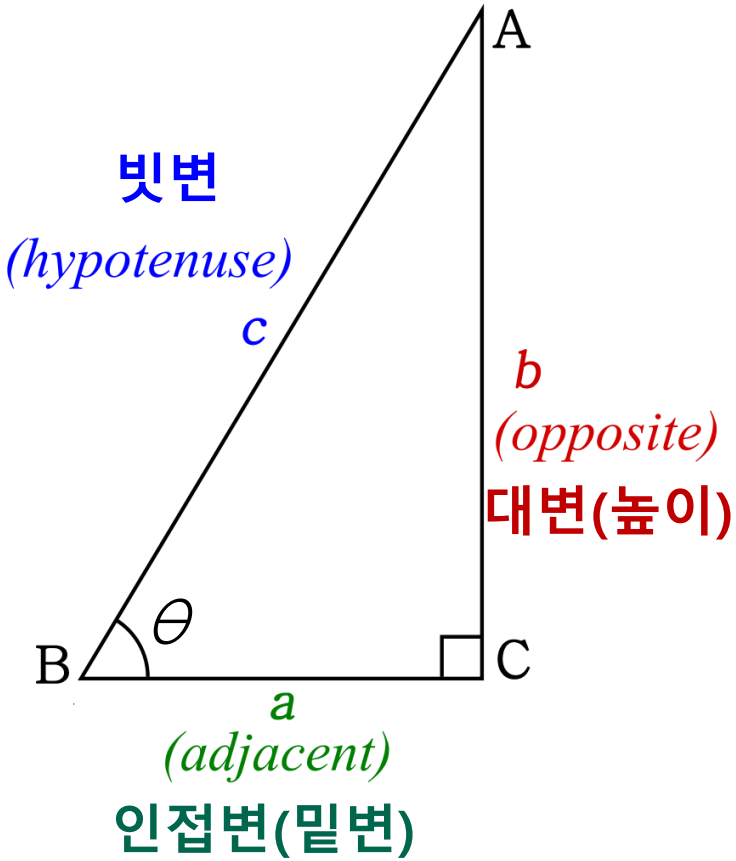
Copyright All rights reserved

4. 삼각함수

회전을 위한 수학

Heesung Oh
Copyright All rights reserved

- **직각 삼각형을 구성하는 세변**



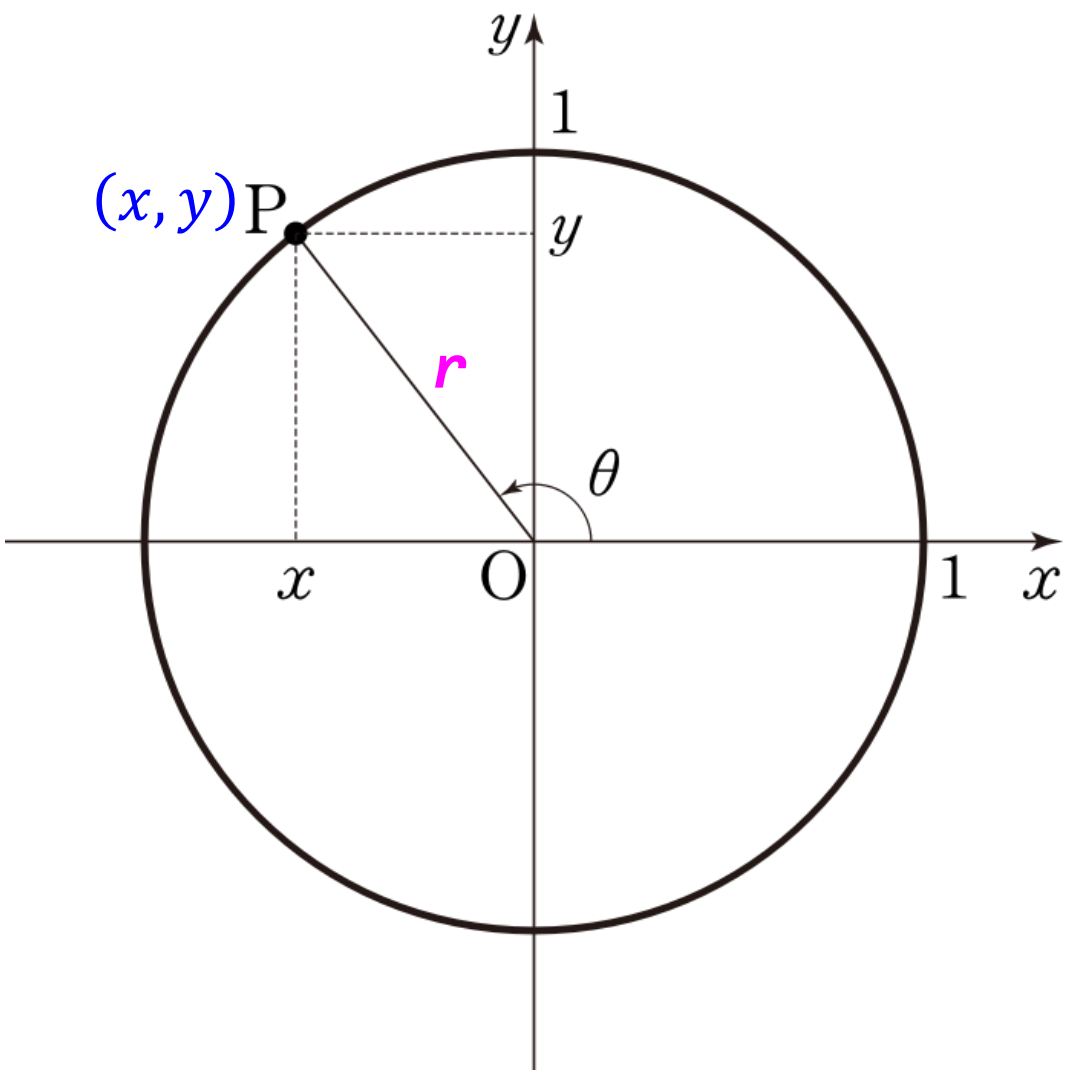
$$\sin\theta = \frac{b}{c}$$

$$\cos\theta = \frac{a}{c}$$

$$\tan\theta = \frac{a}{b}$$

Heesung Oh
Copyright All rights reserved

- **직각 삼각형을 구성하는 세변**



$$\sin\theta = \frac{y}{r}$$

$$\cos\theta = \frac{x}{r}$$

$$\tan\theta = \frac{y}{x}$$

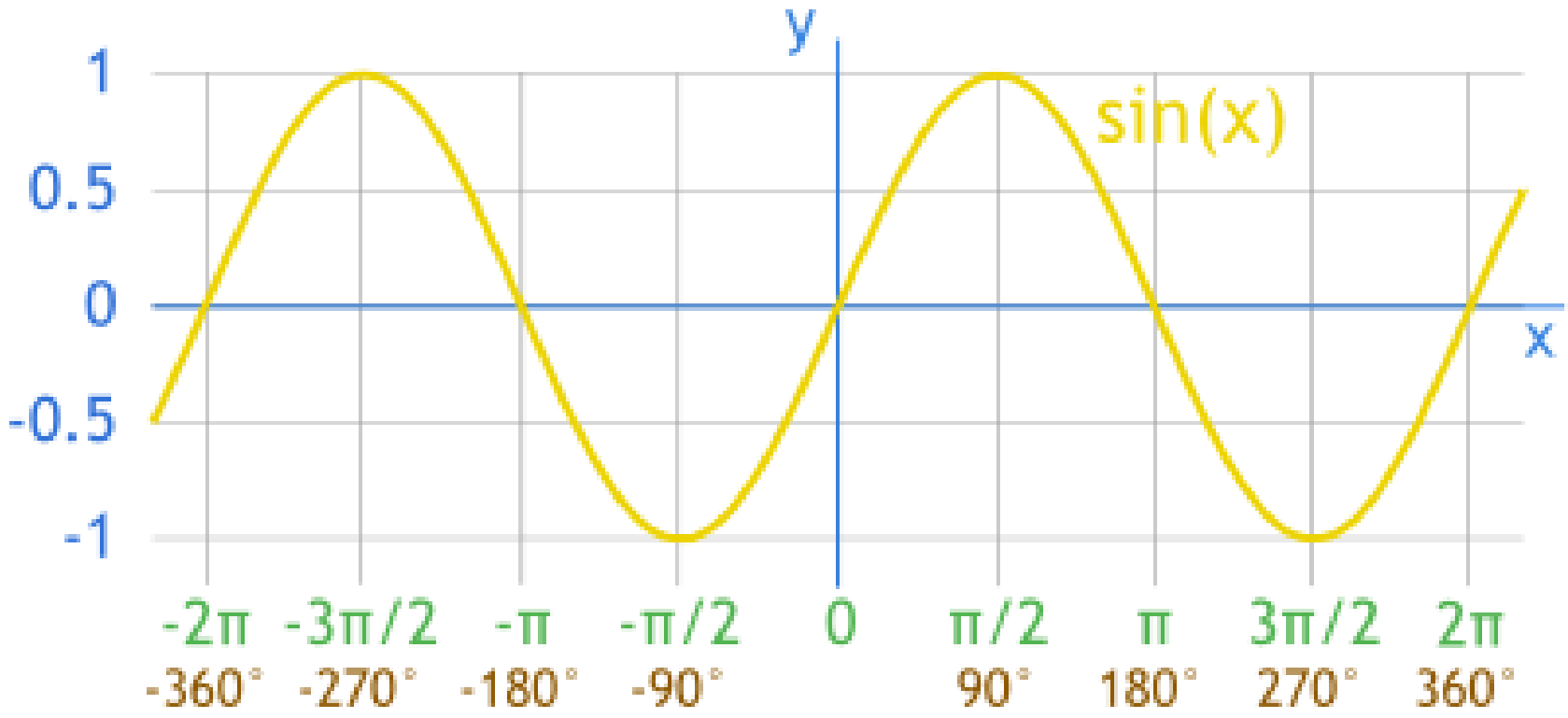
$$(x, y) = (\cos\theta, \sin\theta)$$

$$\cos^2\theta + \sin^2\theta = 1$$

4.1.1 삼각함수의 성질

Heesung Oh
Copyright All rights reserved

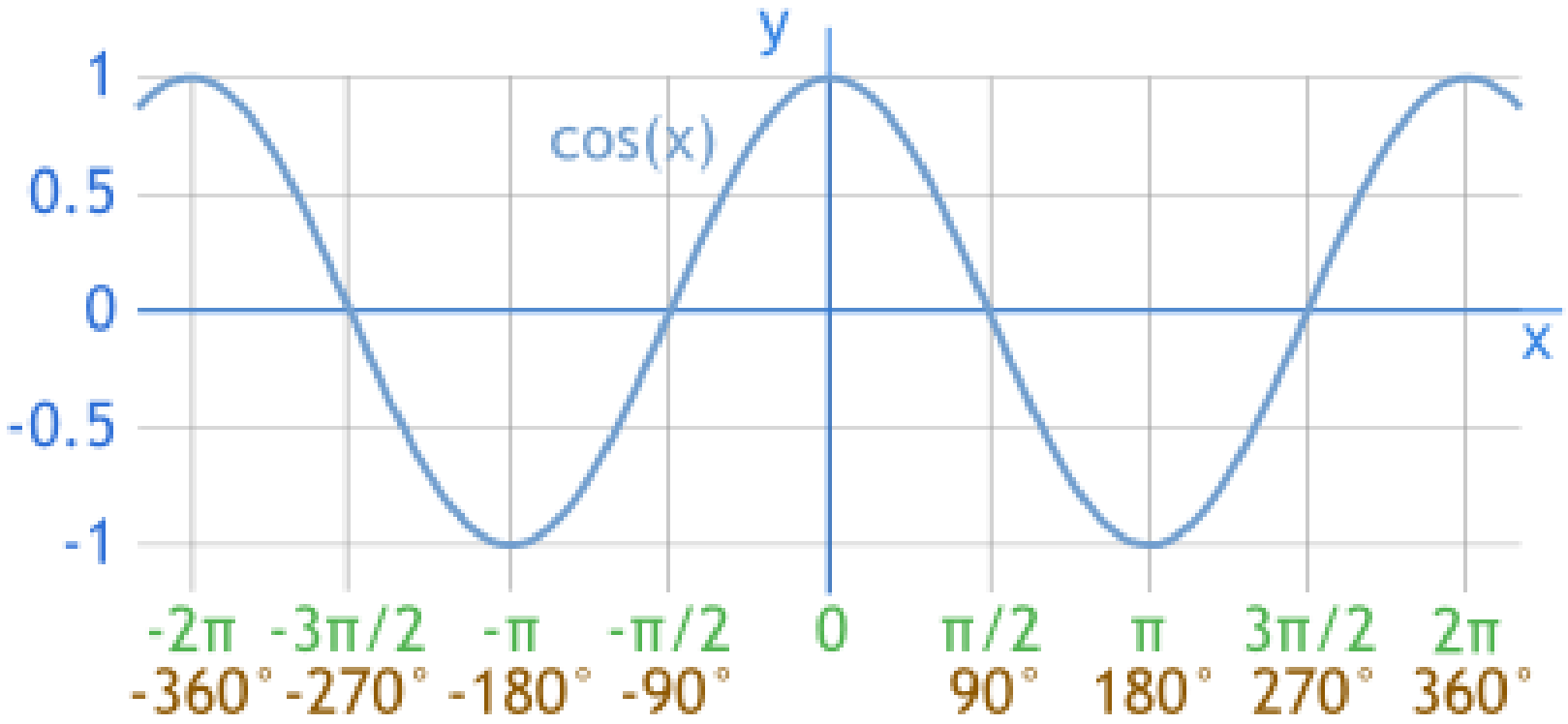
- $\sin(0) = 0$, $\sin(\pi/2) = 1$
- $\sin(\pi) = 0$, $\sin(3\pi/2) = -1$
- $\sin(-x) = -\sin(x)$



4.1.1 삼각함수의 성질

Heesung Oh
Copyright All rights reserved

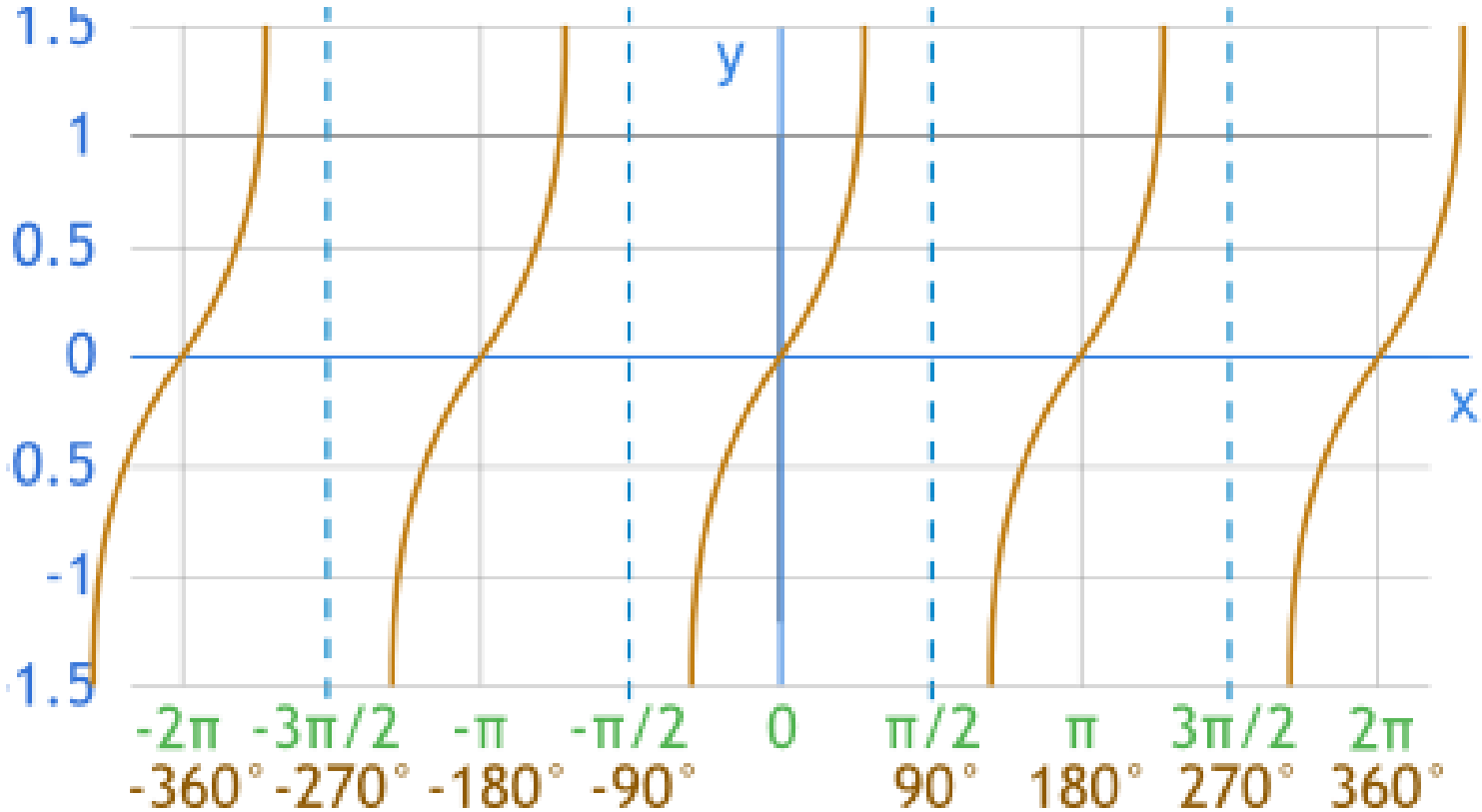
- $\cos(0) = 1$, $\cos(\pi/2) = 0$
- $\cos(\pi) = -1$, $\cos(3\pi/2) = 0$
- $\cos(-x) = \cos(x)$



4.1.1 삼각함수의 성질

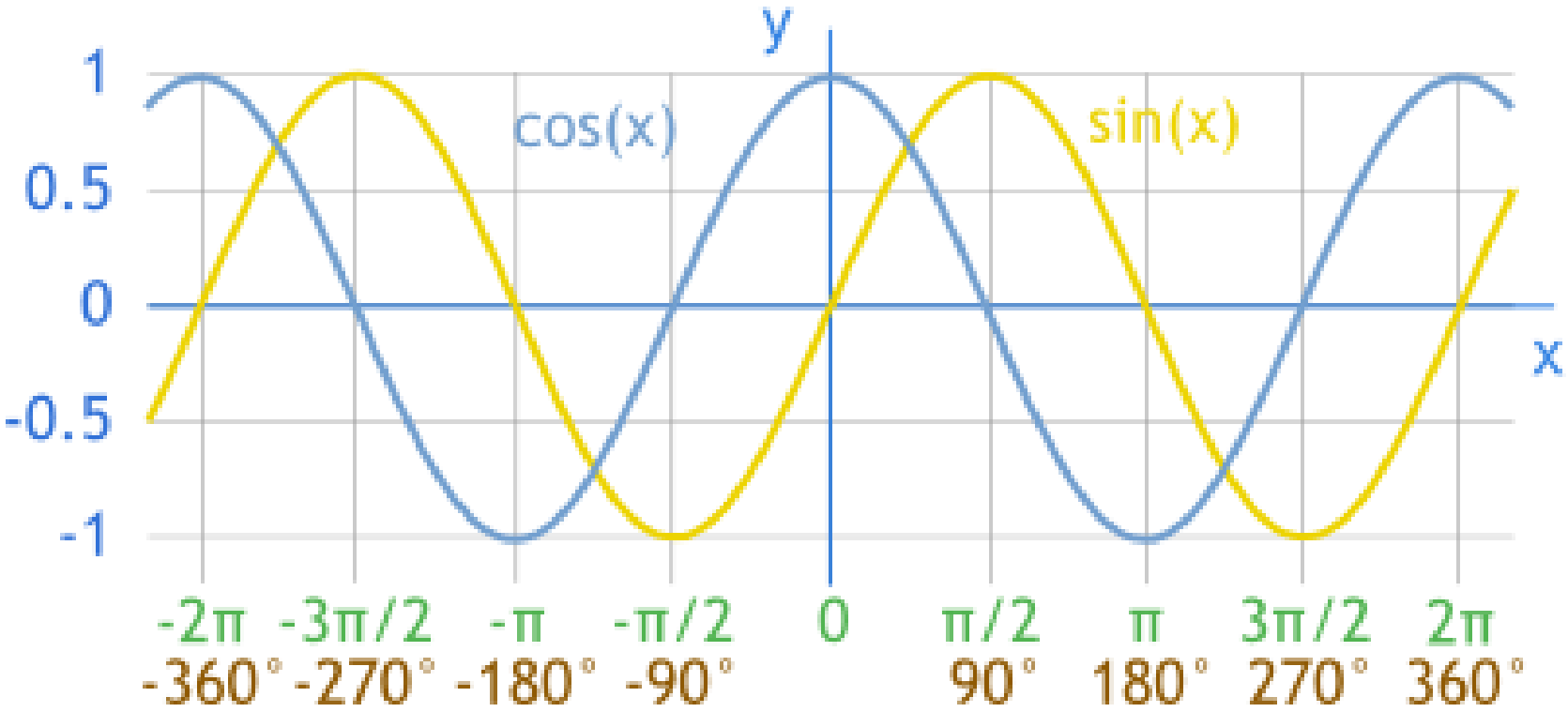
Heesung Oh
Copyright All rights reserved

- $\tan(0) = 1$, $\tan(\pi/2) = \infty$
- $\tan(\pi) = -1$, $\tan(3\pi/2) = -\infty$
- $\tan(-x) = -\tan(x)$



Heesung Oh
Copyright All rights reserved

- sin – cos 함수



4.1.2 각의 측정법

Heesung Oh
Copyright All rights reserved

- Degree
  - [0,360]
  - 달력, 60 진법, 1년 365 → 근사
  - 직관적

- Radian
  - 호의 길이(s)와 반지름(r)의 비율

$$\theta = \frac{s}{r}$$

  - 각도 기준
    - 2π(Rad) = 360° 1rad ≈ 57.3°
    - 1° = π/180 ≈ 0.01745329252, 90° ≈ 1.57079632

각도(°)	라디안(rad)
0°	0
30°	π/6
45°	π/4
60°	π/3
90°	π/2
120°	2π/3
135°	3π/4
180°	π

4.1.2 각의 측정법

Heesung Oh
Copyright All rights reserved

- Radian 장점
  - 각이 곧 길이 → 각도를 바로 거리 계산에 사용 가능
  - 수학적으로 가장 자연 스러운 단위

$$\frac{d}{dx}(\sin x) = \cos x \qquad \frac{d}{dx}(\cos x) = -\sin x$$

  - 작은 각 근사
    - sin$\theta \approx \theta$, tan$\theta \approx \theta$

$$\cos\theta \approx 1 - \frac{\theta^2}{2} \approx 1$$

  - 예)
    - 105mm 포 사거리 10Km 시 편각 1mil = 10m

$$\theta \approx \frac{\text{편 차}}{\text{거 리}}$$

4-1 실습: 삼각함수로 하트 그리기

Heesung Oh
Copyright All rights reserved

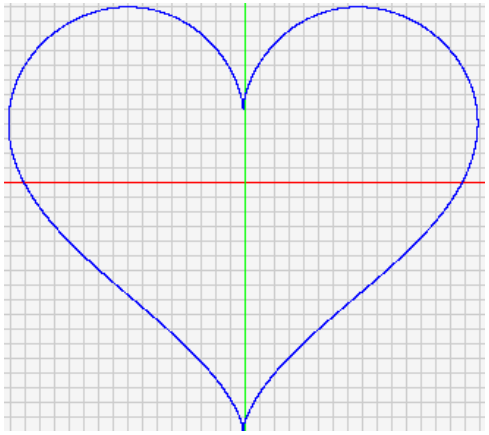
- **Project tree/4-1**

[0, 2π]

$$x = 16\sin^3 x$$

$$y = 13\cos\theta - 5cos2\theta - 2cos3\theta - 2cos4\theta$$

```
float sin = sinf(rad);
float cos = cosf(rad);
float cos2 = cosf(2 * rad);
float cos3 = cosf(3 * rad);
float cos4 = cosf(4 * rad);
float x = 16.f * sin * sin * sin;
float y = 13 * cos - 5 * cos2 - 2 * cos3 - cos4;
```

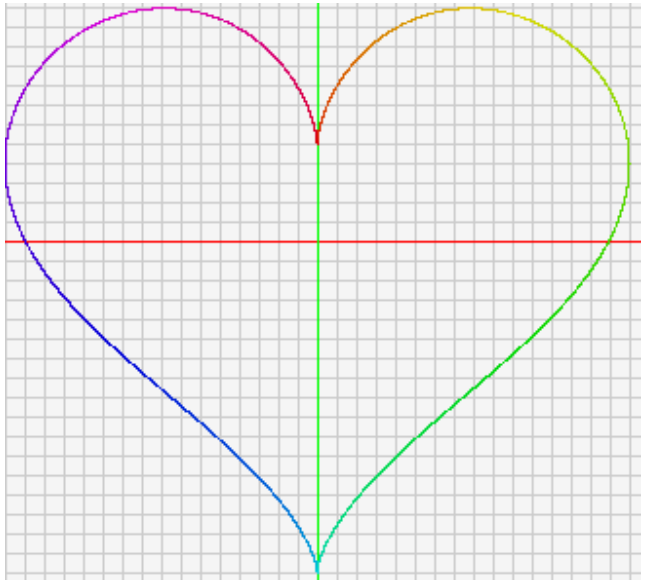


4-2 실습: 하트에 무지개 색상

Heesung Oh
Copyright All rights reserved

- Project tree/4-2

```
struct HSVColor
{
    LinearColor ToLinearColor() const
    {
        ...
        return LinearColor(r, g, b);
    }
    float H = 0.f;
    float S = 1.f;
    float V = 1.f;
};
```



```
hsv.H = rad / Math::TwoPI;
r.DrawPoint(v * currentScale + currentPosition, hsv.ToLinearColor());
```

4-3 실습: 두근 대는 하트

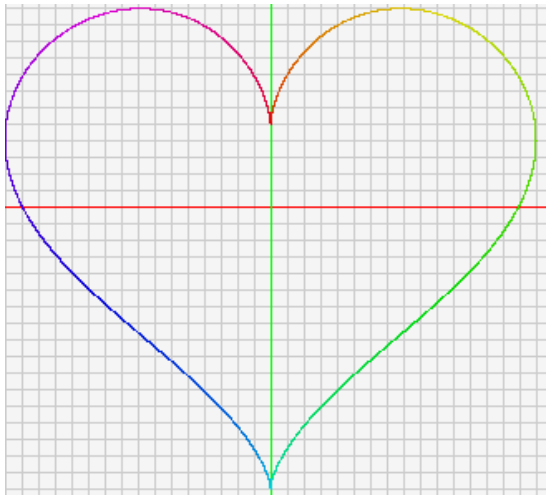
Heesung Oh
Copyright All rights reserved

- **Project tree/4-3**

```
static float duration = 1.5f;
static float elapsedTime = 0.f;
```

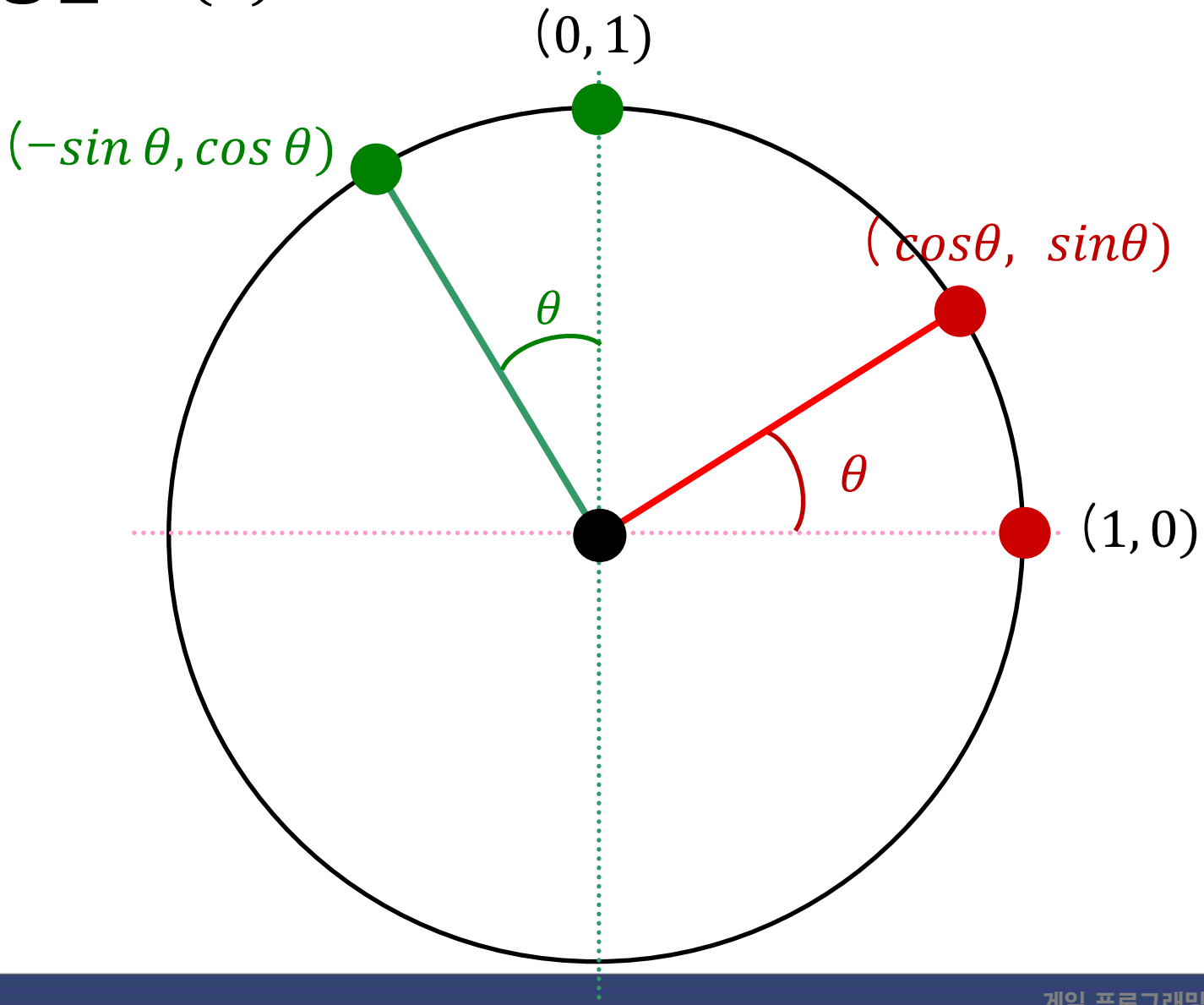
```
// 경과 시간과 사인함수를 활용한 [0,1]값의 생성
elapsedTime += InDeltaSeconds;
elapsedTime = Math::FMod(elapsedTime, duration);
float currentRad = (elapsedTime / duration) * Math::TwoPI;
float alpha = (sinf(currentRad) + 1) * 0.5f;

// 물체의 최종 상태 설정
currentPosition += deltaPosition;
```

```
float alpha = (sinf(currentRad) + 1) * 0.5f;
```



Heesung Oh
Copyright All rights reserved

- 회전 행렬: $R(\theta)$



4.2 삼각함수를 활용한 물체의 회전

Heesung Oh
Copyright All rights reserved

- 회전 행렬: $R(\theta)$

$$R(\theta)(1,\ 0) = (\cos\theta\,, \sin\theta) \qquad R(\theta)\begin{pmatrix}1\\0\end{pmatrix} = \begin{pmatrix}\cos\theta\\ \sin\theta\end{pmatrix}$$

$$R(\theta)(0,\ 1) = (-\sin\theta\,, \cos\theta) \qquad R(\theta)\begin{pmatrix}0\\1\end{pmatrix} = \begin{pmatrix}-\sin\theta\\ \cos\theta\end{pmatrix}$$

$$R(\theta)\begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix} = \begin{pmatrix}\cos\theta & -\sin\theta\\ \sin\theta & \cos\theta\end{pmatrix} \quad \therefore R(\theta) = \begin{pmatrix}cos\,\theta & -sin\,\theta\\ sin\,\theta & cos\,\theta\end{pmatrix}$$

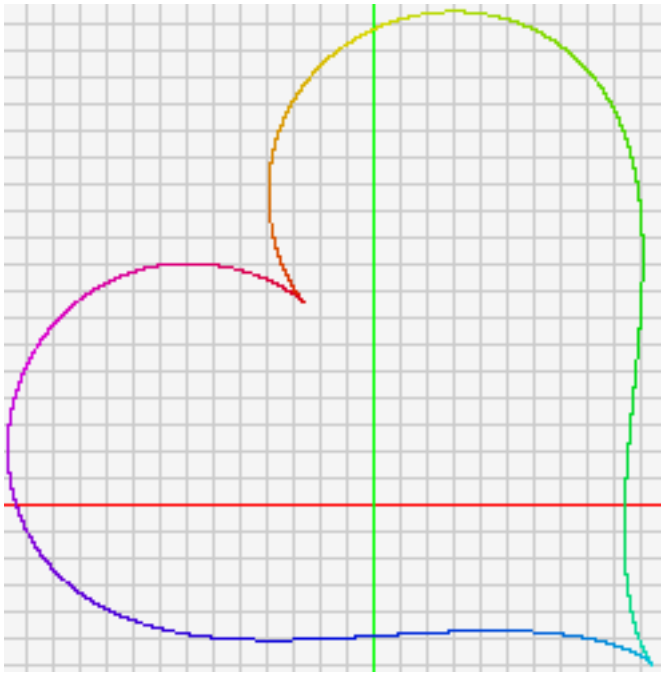
$$\begin{pmatrix}x'\\ y'\end{pmatrix} = \begin{pmatrix}\cos\theta & -\sin\theta\\ \sin\theta & \cos\theta\end{pmatrix}\begin{pmatrix}x\\ y\end{pmatrix} \Rightarrow \begin{aligned} x' &= x\ cos\,\theta - y\ sin\,\theta \\ y' &= x\ sin\,\theta + y\ cos\,\theta \end{aligned}$$

4-4 실습: 하트 회전

Heesung Oh
Copyright All rights reserved

- Project tree/4-4

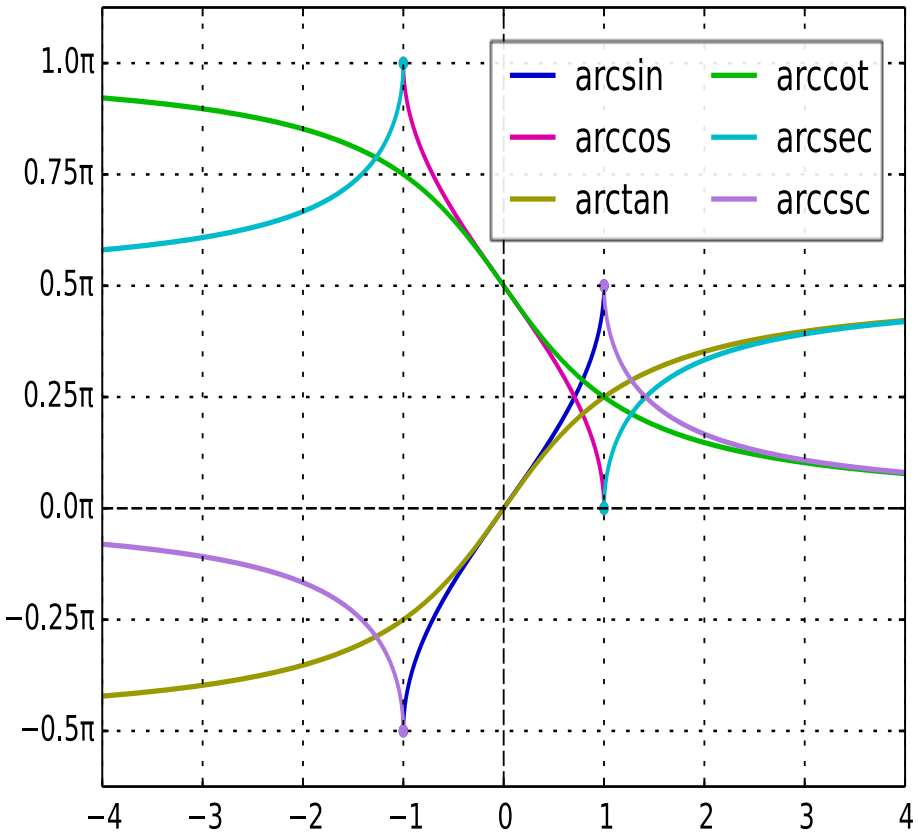
```
Vector2 scaledV = v * currentScale;
// 2. 크기가 변한 점을 회전시킨다.
Vector2 rotatedV = Vector2(scaledV.X * cos - scaledV.Y * sin
                          , scaledV.X * sin + scaledV.Y * cos);
// 3. 회전시킨 점을 이동한다.
Vector2 translatedV = rotatedV + currentPosition;
```



4.3 삼각함수의 역함수

Heesung Oh
Copyright All rights reserved

- sin 역함수
  - $\sin^{-1}(x) = \arcsin(x)$
  - $[-1, 1] \rightarrow [-\pi/2, \pi/2]$
- cos 역함수
  - $\cos^{-1}(x) = \arccos(x)$
  - $[-1, 1] \rightarrow [0, \pi]$
- tan 역함수
  - $\tan^{-1}(x) = \arctan(x)$
    - $[-1, 1] \rightarrow [-\pi/2, \pi/2]$
- arctan2(y,x)
  - $[-1, 1] \rightarrow [-\pi, \pi]$
  - arctan 문제
    - 우측 각도만 구하고 좌/우 구분 불가
    - $x \rightarrow 0$ 이면 에러. arctan2는 안전



Heesung Oh
Copyright All rights reserved

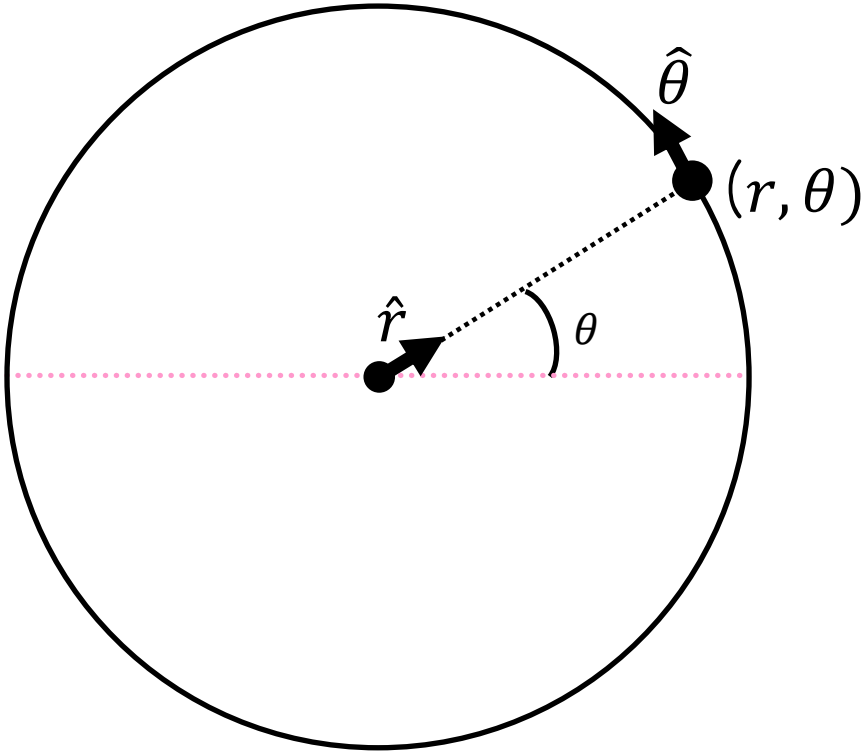
- Polar Coordinate System
  - 원점을 기준으로 평면 상의 위치를 거리와 각도로 표현하는 좌표계

- (x, y) → (r, θ)

$$x = r\cos\theta,\ y = r\sin\theta$$

$$r = \sqrt{x^2 + y^2}$$

$$\theta = arctan2(y, x)$$



Heesung Oh
Copyright All rights reserved

- Vector2

```
float Vector2::Size() const
{
    return ::sqrtf(X * X + Y * Y);
}
```

```
float Vector2::Angle() const
{
    return  ::atan2f(Y,X);
}
```

```
Vector2 Vector2::ToPolarCoordinate() const
{
    return Vector2(Size(), Angle());
}
```

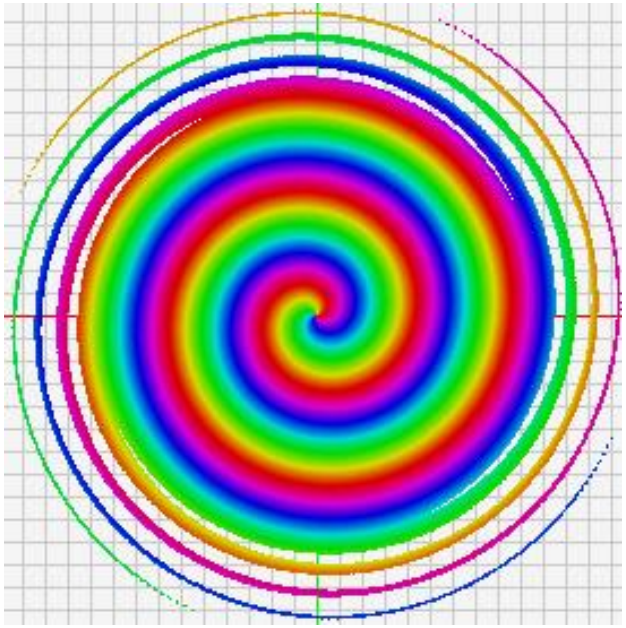
```
Vector2 Vector2::ToCartesianCoordinate() const
{
    float sin = 0.f, cos = 0.f;
    Math::GetSinCosRad(sin, cos, Y);
    return Vector2(X * cos, X * sin);
}
```

4-4 실습: 소용돌이 효과

Heesung Oh
Copyright All rights reserved

- Project tree/4-5

```
// 극 좌표계로 변경
Vector2 polarV = v.ToPolarCoordinate();

// 극좌표계의 각 정보로부터 색상을 결정
if (polarV.Y < 0.f)
{
    polarV.Y += Math::TwoPI;
}
hsv.H = polarV.Y / Math::TwoPI;

// 극좌표계의 크기 정보로부터 회전 크기 결정
float ratio = polarV.X / maxLength;
float weight = Math::Lerp(1.f, 5.f, ratio);

// 극좌표계를 사용해 회전
polarV.Y += Math::Deg2Rad(currentDegree) * weight;

// 최종 값을 데카르트 좌표계로 변환
Vector2 cartesianV = polarV.ToCartesianCoordinate();
```



Copyright All rights reserved

5. 행렬

가상세계의 변환 도구

5.1 선형성: 예측 가능한 비례 관계

Heesung Oh
Copyright All rights reserved

- **선형의 두 조건**
  - **가법성 (Additivity)**
    - 입력의 합 → 출력의 합
    - f(x + y) = f(x) + f(y)

$$f\left(\sum x_i\right) = \sum f(x_i)$$

  - **1차 동차성 (Homogeneity)**
    - 입력의 스칼라 배 → 출력의 동일한 스칼라 배
    - $f(\alpha x) = \alpha f(x)$

  - $f(\alpha x + \beta y) = \alpha f(x) + \beta f(y)$

$$f\left(\sum a_i \cdot x_i\right) = \sum a_i \cdot f(x_i)$$

5.1.1 선형함수

Heesung Oh
Copyright All rights reserved

- **선형 함수**
  - 직선함수: $f(x) = \alpha x$, 또는 $y = \alpha x$

- **$f(x) = \alpha x$ 가 선형함수 인지 증명**

$$f(x_1 + x_2) = \alpha(x_1 + x_2)$$
$$= \alpha x_1 + \alpha x_2$$
$$= f(x_1) + f(x_2)$$
$$f(\kappa x) = \alpha(\kappa x) = \kappa(\alpha x) = \kappa f(x)$$

5.1.1 선형함수

Heesung Oh
Copyright All rights reserved

- $f(x) = \alpha x^2$ 는 선형함수?

  $f(x_1 + x_2) = \alpha(x_1 + x_2)^2$
  $= \alpha x_1^2 + \alpha x_2^2 + 2\alpha x_1 x_2$
  $= f(x_1) + f(x_2) + 2\alpha x_1 x_2$ → 선형이 아님

- $f(x) = \alpha x + \beta$ 가 선형함수 인지 증명

  $f(x_1 + x_2) = \alpha(x_1 + x_2) + \beta$
  $= \alpha x_1 + \alpha x_2 + \beta$
  $= f(x_1) + f(x_2) - \beta$ → 선형이 아님

5.1.2 벡터 공간의 선형 변환

Heesung Oh
Copyright All rights reserved

- 벡터 공간의 선형 변환
  - 선형 변환 함수 $f(x)$ 에 대해서 $f(\vec{v})$ 도 선형?

- 2차원 벡터에서 증명 (x, y)
  - $f(\vec{v}) = (ax + by,\ cx + dy)$

- 가법성

$$
\begin{aligned}
f(\overrightarrow{v_1} + \overrightarrow{v_2}) &= f(x_1 + x_2, x_1 + x_2) \\
&= \big(a(x_1 + x_2) + b(y_1 + y_2),\ c(x_1 + x_2) + d(y_1 + y_2)\big) \\
&= (ax_1 + ax_2 + by_1 + by_2,\ cx_1 + cx_2 + dy_1 + dy_2) \\
&= (ax_1 + by_1 + ax_2 + by_2,\ cx_1 + dy_1 + cx_2 + dy_2) \\
&= (ax_1 + by_1,\ cx_1 + dy_1) + (ax_2 + by_2,\ cx_2 + dy_2) \\
&= f(\overrightarrow{v_1}) + f(\overrightarrow{v_2})
\end{aligned}
$$

5.1.2 벡터 공간의 선형 변환

Heesung Oh
Copyright All rights reserved

- 스칼라 곱
  - $f(k\vec{v}) = \big(k(ax+by),\ k(cx+dy)\big)$

- 증명

$$
\begin{aligned}
f(k\vec{v}) &= \big(k(ax+by),\ k(cx+dy)\big) \\
&= k\big((ax+by),\ (cx+dy)\big) \\
&= k(ax+by,\ cx+dy) \\
&= kf(\vec{v})
\end{aligned}
$$

- 회전 변환: 선형 변환

$$
\begin{aligned}
x' &= x\ \cos\theta - y\ \sin\theta \\
y' &= x\ \sin\theta + y\ \cos\theta
\end{aligned}
\qquad
\begin{matrix}
\cos\theta & -\sin\theta \\
\sin\theta & \cos\theta
\end{matrix}
$$

5.2 행렬

Heesung Oh
Copyright All rights reserved

- 행렬(Matrix)
  - m개의 행(Row)과 n개의 열(Colum)로 구성된 직사각형 형태의 수 배열
  - $\begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix}$ $\quad \boldsymbol{M_{mn}} = \begin{pmatrix} M_{11} & \cdots & M_{1n} \\ \vdots & \ddots & \vdots \\ M_{m1} & \cdots & M_{mn} \end{pmatrix}$

- 행벡터 (Row Vector)
  - 하나의 행(row)으로 이루어진 행렬
  - 1 × n
  - $[\, a_1 \;\; a_2 \cdots a_n \,]$

- 열벡터 (Column Vector)
  - n × 1
  - 하나의 열(column)로 이루어진 행렬

$$\begin{pmatrix} a_1 \\ a_2 \\ \cdots \\ a_n \end{pmatrix}$$

Heesung Oh
Copyright All rights reserved

- **정방 행렬 (Square Matrix)**
  - 행과 열의 크기가 같은 n X n 행렬

$$\boldsymbol{M_{nn}} = \begin{pmatrix} m_{11} & \cdots & m_{1n} \\ \vdots & \ddots & \vdots \\ m_{n1} & \cdots & m_{nn} \end{pmatrix}$$

$$M_{row\,1} = (m_{11} \quad \cdots \quad m_{1n})$$

$$M_{row\,n} = (m_{n1} \quad \cdots \quad m_{nn})$$

$$M_{col\,1} = (m_{11} \quad \cdots \quad m_{n1})$$

$$M_{col\,1} = (m_{1n} \quad \cdots \quad m_{nn})$$

5.2.1 행렬의 기본 연산

Heesung Oh
Copyright All rights reserved

- **행렬의 덧셈**
  - 같은 차원에서만 가능

$$\boldsymbol{A}+\boldsymbol{B}=\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}+\begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}=\begin{pmatrix} a_{11}+b_{11} & a_{12}+b_{12} \\ a_{21}+b_{21} & a_{22}+b_{22} \end{pmatrix}$$

- **스칼라 곱**

$$k\cdot\boldsymbol{A}=k\cdot\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}=\begin{pmatrix} k\cdot a_{11} & k\cdot a_{12} \\ k\cdot a_{21} & k\cdot a_{22} \end{pmatrix}$$

- **전치 (Transpose)**

$$M_{mn}^{T}=M_{nm}$$

Heesung Oh
Copyright All rights reserved

- **행렬의 곱셈**
  - 행렬의 곱셈은 앞 행렬 열 개수와 뒤 행렬 행 개수 같을 때, 앞 행렬 각 행과 뒤 행렬 각 열 내적으로 정의

$$C_{ij} = \sum A_{i\,k} \cdot B_{k\,j} \qquad C_{ij} = A_{i\,k} \cdot B_{k\,j}$$

Heesung Oh
Copyright All rights reserved

행렬의 곱셈

결합 법칙

$$
\begin{aligned}
M &= (AB)C \\
&= \sum\left(\sum A_{i\,l}\cdot B_{l\,k}\right)\cdot C_{k\,j} \\
&= \sum\sum A_{i\,l}\cdot B_{l\,k}\cdot C_{k\,j} \\
&= \sum\sum A_{i\,l}\cdot\left(B_{l\,k}\cdot C_{k\,j}\right) \\
&= \sum A_{i\,l}\cdot(BC)_{l\,j} \\
&= A(BC)
\end{aligned}
$$

Heesung Oh
Copyright All rights reserved

행렬의 곱셈

교환법칙

$$(AB) = A_{i\,k} \cdot B_{k\,j}$$
$$(BA) = B_{i\,k} \cdot A_{k\,j}$$
$$(AB) \neq (BA)$$

전치

$$\begin{aligned}(AB)^t &= \left(AB_{i\,j}\right)^t \\ &= (AB)_{j\,i} = A_{j\,k} \cdot B_{k\,i} \\ &= B_{k\,i} \cdot A_{j\,k} = B_{i\,k}{}^t \cdot A_{k\,j}{}^t \\ &= B^t A^t\end{aligned}$$

$$C_{ij} = A_{i\,k} \cdot B_{k\,j}$$

Heesung Oh
Copyright All rights reserved

- **벡터 행렬의 곱셈**

$$B \cdot A \cdot \vec{v} = \vec{w}$$
$$(B \cdot A \cdot \vec{v})^t = (\vec{w})^t$$
$$\vec{v}^t \cdot A^t \cdot B^t = (\vec{w})^t$$

Heesung Oh
Copyright All rights reserved

- **2차원 벡터**
  - **기저 벡터로 구성한 행렬로 표기**

$$\vec{v} = x\hat{e}_1 + y\hat{e}_2 \qquad \hat{e}_1 = (e_{1x}\hat{x} + e_{1y}\hat{y}), \hat{e}_2 = (e_{2x}\hat{x} + e_{2y}\hat{y})$$

$$= (xe_{1x}\hat{x} + xe_{1y}\hat{y}) + (ye_{2x}\hat{x} + ye_{2y}\hat{y})$$

$$= (xe_{1x} + ye_{2x})\hat{x} + (xe_{1y} + ye_{2y})\hat{y}$$

$$= (xe_{1x} + ye_{2x},\ xe_{1y} + ye_{2y})$$

$$= \begin{pmatrix} e_{1x} & e_{2x} \\ e_{1y} & e_{2y} \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

$$= (\hat{e}_1,\ \hat{e}_2) \begin{pmatrix} x \\ y \end{pmatrix} \qquad \begin{pmatrix} e_{1x} & e_{2x} \\ e_{1y} & e_{2y} \end{pmatrix} \quad (\hat{e}_1,\ \hat{e}_2)$$

5.3.1 크기 변환행렬

Heesung Oh
Copyright All rights reserved

- **크기 변환 행렬 (Scaling Matrix)**

$$\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}$$

- **회전 변환 → 크기 변환 ≠ 크기 변환 → 회전 변환**

$$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix} = \begin{pmatrix} s_x\cos\theta & -s_y\sin\theta \\ s_x\sin\theta & s_y\cos\theta \end{pmatrix}$$

$$\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} = \begin{pmatrix} s_x\cos\theta & -s_x\sin\theta \\ s_y\sin\theta & s_y\cos\theta \end{pmatrix}$$

$$R_\theta \times (S_x, S_y) \neq (S_x, S_y) \times R_\theta$$

Heesung Oh
Copyright All rights reserved

- **회전 변환행렬**

$$R_{\frac{\pi}{2}} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \qquad \overrightarrow{v'} = (-y,\ x)$$

$$R_{-\frac{\pi}{2}} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \qquad \overrightarrow{v'} = (y,\ -x)$$

$$R_{\theta} = \begin{pmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{pmatrix}$$

5.3.3 전단 변환행렬

Heesung Oh
Copyright All rights reserved

- **전단 변환행렬**

$$S = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$$

Heesung Oh
Copyright All rights reserved

- **삼각함수의 덧셈 정리**

$$\cos(\alpha+\beta) = \cos\alpha\cos\beta - \sin\alpha\sin\beta$$
$$\sin(\alpha+\beta) = \sin\alpha\cos\beta + \cos\alpha\sin\beta$$

$$R_\alpha = \begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix} \quad R_\beta = \begin{pmatrix} \cos\beta & -\sin\beta \\ \sin\beta & \cos\beta \end{pmatrix}$$

$$\begin{aligned} R_\beta R_\alpha &= \begin{pmatrix} \cos\beta & -\sin\beta \\ \sin\beta & \cos\beta \end{pmatrix}\begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix} \\ &= \begin{pmatrix} \cos\beta\cos\alpha - \sin\beta\sin\alpha & -\cos\beta\sin\alpha - \sin\beta\cos\alpha \\ \sin\beta\cos\alpha + \cos\beta\sin\alpha & -\sin\beta\sin\alpha + \cos\beta\cos\alpha \end{pmatrix} \\ &= \begin{pmatrix} \cos\alpha\cos\beta - \sin\alpha\sin\beta & -\sin\alpha\cos\beta - \cos\alpha\sin\beta \\ \sin\alpha\cos\beta + \cos\alpha\sin\beta & \cos\alpha\cos\beta - \sin\alpha\sin\beta \end{pmatrix} \\ &= \begin{pmatrix} \cos(\alpha+\beta) & -\sin(\alpha+\beta) \\ \sin(\alpha+\beta) & \cos(\alpha+\beta) \end{pmatrix} \\ &= R_{\alpha+\beta} \end{aligned}$$

5-1 실습: 하트의 크기 변환 회전 변환

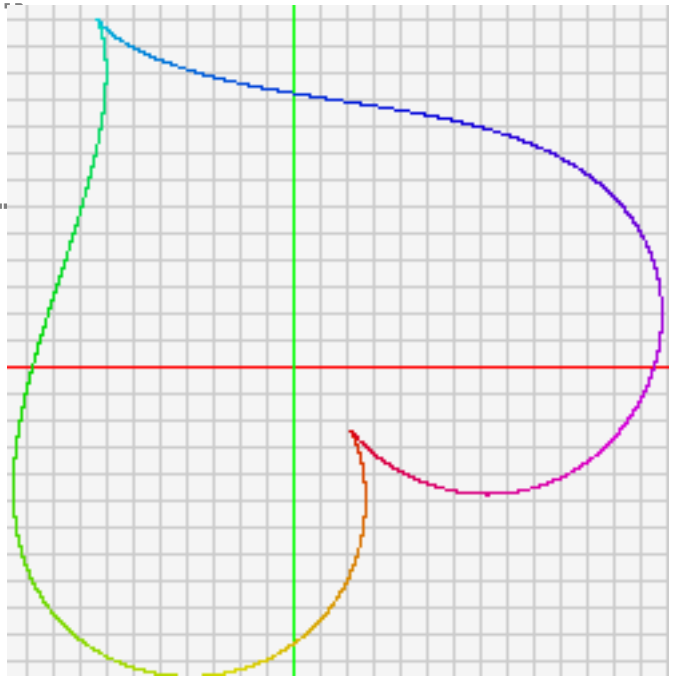
Heesung Oh
Copyright All rights reserved

- **Project tree/5-1**

```
Matrix2x2 operator*(float) const;
Matrix2x2 operator*(const Matrix2x2&) const;
Vector2 operator*(const Vector2&) const;
friend Vector2 operator*=(Vector2&, const Matrix2x2&)
```

```
// 회전 변환 행렬의 기저 벡터와 행렬
Vector2 rBasis1(cos, sin);
Vector2 rBasis2(-sin, cos);
Matrix2x2 rMatrix(rBasis1, rBasis2);

// 크기 변환 행렬의 기저 벡터와 행렬
Vector2 sBasis1 = Vector2::UnitX * currentScale;
Vector2 sBasis2 = Vector2::UnitY * currentScale;
Matrix2x2 sMatrix(sBasis1, sBasis2);

// 크기, 회전의 순서로 진행하는 합성 변환 행렬의 계산
Matrix2x2 finalMatrix = rMatrix * sMatrix;
```

```
// 1. 점에 행렬을 적용한다.
Vector2 transformedV = finalMatrix * v;
// 2. 변환된 점을 이동한다.
Vector2 translatedV = transformedV + currentPosition;
```



5.4 역행렬

Heesung Oh
Copyright All rights reserved

- **항등행렬**

$$I_{nn} = 1$$

- **역행렬**
  - 행렬에 곱했을 때 단위행렬이 되는 행렬
  - $A A^{-1} = A^{-1} A = I$

5.4.1 역행렬 존재 판별식

Heesung Oh
Copyright All rights reserved

- **역행렬 존재**

$$M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

det(M) = ad −bc ≠ 0

$$M^{-1} = \frac{1}{det(M)} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

5.4.2 크기 변환행렬의 역행렬

Heesung Oh
Copyright All rights reserved

- **크기 변환 행렬 (Scaling Matrix)**

$$S = \begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}$$

- **역행렬**

$$S^{-1} = \begin{pmatrix} \frac{1}{s_x} & 0 \\ 0 & \frac{1}{s_y} \end{pmatrix}$$

5.4.3 전단 변환행렬의 역행렬

Heesung Oh
Copyright All rights reserved

- **전단 행렬**

$$S = \begin{pmatrix} 1 & a \\ 0 & 1 \end{pmatrix}$$

- **전단행렬 역행렬**

$$S = \begin{pmatrix} 1 & -a \\ 0 & 1 \end{pmatrix}$$

5.4.4 회전 변환행렬의 역행렬

Heesung Oh
Copyright All rights reserved

- **회전행렬**

$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

- **회전행렬 역행렬**

$$R^{-1} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix}$$

$$= R_{(-\theta)} = R^t$$

Heesung Oh
Copyright All rights reserved

- **행렬곱의 역행렬**

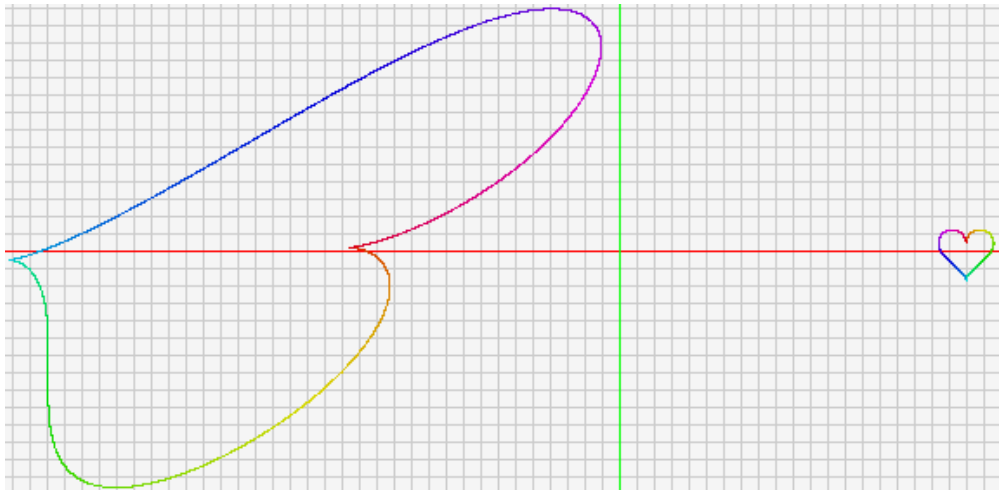
$$I = A \cdot A^{-1} = A \cdot I \cdot A^{-1}$$
$$I = A \cdot B \cdot B^{-1} \cdot A^{-1}$$
$$I = (A \cdot B) \cdot (B^{-1} \cdot A^{-1})$$
$$= (A \cdot B) \cdot (A \cdot B)^{-1}$$

$$\therefore (A \cdot B)^{-1} = B^{-1} \cdot A^{-1}$$

Heesung Oh
Copyright All rights reserved

- Project tree/5-2

```
// 크기 변환 행렬
Vector2 sBasis1(currentScale, 0.f);
Vector2 sBasis2(0.f, currentScale);
Matrix2x2 sMatrix(sBasis1, sBasis2);

// 회전 변환 행렬
float sin, cos;
Math::GetSinCos(sin, cos, currentDegree);
Vector2 rBasis1(cos, sin);
Vector2 rBasis2(-sin, cos);
Matrix2x2 rMatrix(rBasis1, rBasis2);

// 전단 변환 행렬
Vector2 shBasis1 = Vector2::UnitX;
Vector2 shBasis2(currentShear, 1.f);
Matrix2x2 shMatrix(shBasis1, shBasis2);

// 합성 행렬
Matrix2x2 cMatrix = shMatrix * rMatrix * sMatrix;

// 크기 변환 행렬의 역행렬
float invScale = 1.f / currentScale;
Vector2 isBasis1(invScale, 0.f);
Vector2 isBasis2(0.f, invScale);
Matrix2x2 isMatrix(isBasis1, isBasis2);

// 회전 변환 행렬의 역행렬
Matrix2x2 irMatrix = rMatrix.Transpose();

// 전단 변환 행렬의 역행렬
Vector2 ishBasis1 = Vector2::UnitX;
Vector2 ishBasis2(-currentShear, 1.f);
Matrix2x2 ishMatrix(ishBasis1, ishBasis2);

// 역행렬의 합성행렬. ( 역순으로 조합하기 )
Matrix2x2 icMatrix = isMatrix * irMatrix * ishMatrix;
```



```
// 왼쪽 하트 ( 변환 행렬을 적용하기 )
Vector2 left = cMatrix * v;
r.DrawPoint(left - pivot, hsv.ToLinearColor());

// 오른쪽 하트 ( 왼쪽 하트에 추가로 역행렬을 적용하기 )
Vector2 right = icMatrix * left;
r.DrawPoint(right + pivot, hsv.ToLinearColor());
```

Copyright All rights reserved

6. 아핀 공간

움직이는 가상 세계의 구축

6.1 이동 변환을 위한 아핀 공간

Heesung Oh
Copyright All rights reserved

- **아핀 공간의 필요성**

$$\begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} x + a \\ y + b \end{pmatrix}$$

수식을 만족하는
2x2 정방 행렬 M 없음

$$M \cdot \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x + a \\ y + b \end{pmatrix}$$

- **동차 변환 행렬을 사용하면 표현 가능**

$$\begin{pmatrix} 1 & 0 & a \\ 0 & 1 & b \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + a \\ y + b \\ 1 \end{pmatrix}$$

6.1 이동 변환을 위한 아핀 공간

Heesung Oh
Copyright All rights reserved

- **아핀 공간**
  - 아핀 공간은 점과 벡터의 관계(연산)로 정의되는 공간

- **점(위치) + 벡터 → 새로운 점(위치)**

$$P + \vec{v} = Q$$

- **동차 좌표 표현(3차원)**
  - 위치: (x, y, z, 1)
  - 벡터: (x, y, z, 0)

6.1 이동 변환을 위한 아핀 공간

Heesung Oh
Copyright All rights reserved

- 아핀 변환
  - 선형 변환 + 평행 이동
  - 직선 유지
  - 평행성 유지
  - 길이 / 각도 보존 X

- 아핀 변환 행렬
  - 선형 변환 행렬 + 이동 벡터 구조

- 형태 (3D)
  - $x' = R \cdot x + T$
  - R → 회전 / 스케일 / 전단
  - T → 평행 이동

Heesung Oh
Copyright All rights reserved

- **크기/회전/이동 변환 행렬 (Translate Transform Matrix)**

$$R = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \qquad S = \begin{pmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$T = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

$$T \cdot R \cdot S = \begin{pmatrix} S_x \cos\theta & -S_y \sin\theta & t_x \\ S_x \sin\theta & S_y \cos\theta & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

Heesung Oh
Copyright All rights reserved

- Project tree/6-1

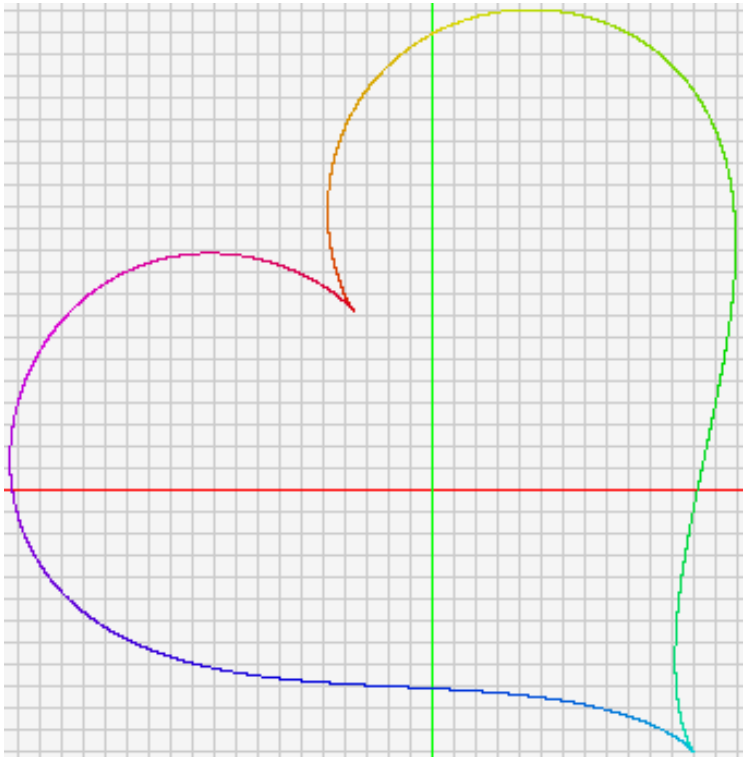
```
// 아핀 변환 행렬 ( 크기 )
Vector3 sBasis1(currentScale, 0.f, 0.f);
Vector3 sBasis2(0.f, currentScale, 0.f);
Vector3 sBasis3 = Vector3::UnitZ;
Matrix3x3 sMatrix(sBasis1, sBasis2, sBasis3);
```

```
// 아핀 변환 행렬 ( 회전 )
float sin = 0.f, cos = 0.f;
Math::GetSinCos(sin, cos, currentDegree);
Vector3 rBasis1(cos, sin, 0.f);
Vector3 rBasis2(-sin, cos, 0.f);
Vector3 rBasis3 = Vector3::UnitZ;
Matrix3x3 rMatrix(rBasis1, rBasis2, rBasis3);
```

```
// 아핀 변환 행렬 ( 이동 )
Vector3 tBasis1 = Vector3::UnitX;
Vector3 tBasis2 = Vector3::UnitY;
Vector3 tBasis3(currentPosition.X, currentPosition.Y, 1.f);
Matrix3x3 tMatrix(tBasis1, tBasis2, tBasis3);
```

```
// 모든 아핀 변환을 곱한 합성 행렬. 크기-회전-이동 순으로 적용
Matrix3x3 finalMatrix = tMatrix * rMatrix * sMatrix;
```

```
// 곱셈을 위해 하트를 구성하는 벡터를 3차원으로 변경
Vector3 newV(v.X, v.Y, 1.f);

// 아핀 변환을 적용하고 마지막 차원의 값을 제거하고 사용
Vector3 finalV = finalMatrix * newV;
```



Heesung Oh
Copyright All rights reserved

- 6.2.1 점
  - 마지막 차원 값이 1인 아핀 공간의 원소를 점(Point)
  - 행렬 곱을 사용, 이동 가능
  - 가상 공간에서 이동은 점을 사용

  - 2차원 점: (x, y, 1)
    - (x, y, 2) → (x + 2a, y + 2b , 2)

$$\begin{pmatrix} 1 & 0 & a \\ 0 & 1 & b \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 2 \end{pmatrix} = \begin{pmatrix} x + 2a \\ y + 2b \\ 1 \end{pmatrix}$$

  - 3차원 점: (x, y, z, 1)

Heesung Oh
Copyright All rights reserved

- 6.2.2 이동 벡터
  - 아핀 공간에 닫혀 있음
  - 아핀 공간 벡터 == 이동 벡터, 또는 변위 벡터(Displacement vector)

$$P_1 + \vec{v} = P_2$$

$$\vec{v} = P_2 - P_1 = (x_2 - x_1, y_2 - y_1, 0)$$

6.2 아핀 공간의 구성 요소

Heesung Oh
Copyright All rights reserved

- 점과 벡터 정의
  - 점: 물체를 표현하고 위치를 지정
    - 동차 좌표 차원 값 = 1
  - 이동 벡터: 물체의 이동에 사용
    - 동차좌표 차원 값 = 0
  - 정리
    - 점 – 점 = 벡터
    - 점 +벡터 = 점
    - 벡터 + 벡터 = 벡터
    - 점 + 점 ≠ 점

$$\vec{v} = P - 0$$
$$\vec{v} = (x, y, 1) - (0,0,1) = (x, y, 0)$$

- 0 벡터
  - 아핀 공간에서 같은 점들의 차
  - 2차원: (0,0,1), 3차원: (0,0,0,1)

Heesung Oh
Copyright All rights reserved

- 6.2.4 아핀 공간 특징
  - 점: 차원을 하나 늘리고 마지막 값 → 1
    - ex) 2차원: (x,y,1) , n차원: (p_1, ⋯ p_n, 1)
  - 벡터 → 변위 벡터: 마지막 값 → 0

6.3 아핀 결합

Heesung Oh
Copyright All rights reserved

- **두 점 사이의 선형 결합**

$$aP_1 + bP_2 = (ax_1 + bx_2,\ ay_1 + by_2,\ a + b)$$

  - $a + b = 1$ **이면 점의 생성 가능**

- **아핀 결합**
  - **아핀 공간에서 점의 스칼라 곱과 점 사이 덧셈은 정의 안됨**
  - **단, 계수의 합이 1인 경우에 한해 점들의 결합이 허용**

$$\sum c_i \cdot P_i = c_1 \cdot P_1 + c_2 \cdot P_2 + \cdots + c_n \cdot P_n \quad \Leftarrow \quad \sum c_i = 1$$

Heesung Oh
Copyright All rights reserved

- **아핀 결합 유도**
  - 점의 스칼라 곱, 점 + 점의 덧셈 연산을 잊어야
  - 오직 수식으로만 이해
  - P_o: 원점이 아닌 기준점

$$
\begin{aligned}
P_\exists &= P_0 + \vec{D} \\
&= P_0 + \sum c_i \cdot \vec{v_i} \\
&= P_0 + \sum c_i \cdot (P_i - P_0) \\
&= P_0 (1 - c_1 - c_2 \cdots - c_n) + \sum c_i \cdot P_i
\end{aligned}
$$

$$\vec{v_i} = P_i - P_0$$

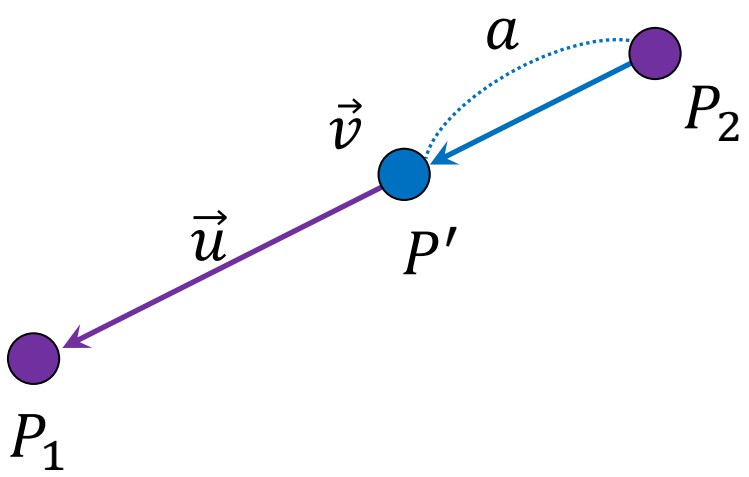
$$1 - c_1 - c_2 \cdots - c_n = 0 \Rightarrow c_1 + c_2 \cdots + c_n = 1$$

$$P_\exists = \sum c_i \cdot P_i \Leftarrow \text{아핀 결합} \qquad P = \frac{\sum c_i \cdot P_i}{\sum c_i}$$

Heesung Oh
Copyright All rights reserved

- **두 점의 결합**
  - 선형 결합: $a + b = 1$

    $$aP_1 + (1-a)P_2 = P'$$



  - $a = 1$

    $$1 \cdot P_1 + 0 = P_1$$

  - $a = 0$

    $$0 + 1 \cdot P_1 = P_2$$

  - **직선의 방정식**

    $$a(P_1 - P_2) = (P' - P_2)$$

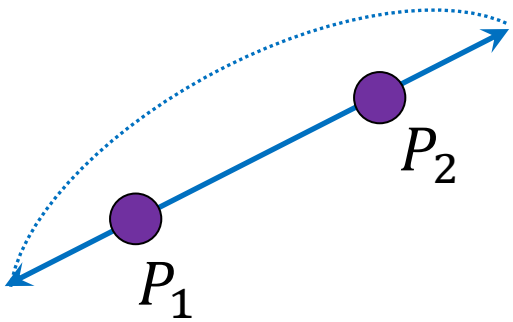
    $$a \cdot \vec{u} = \vec{v}$$

    $$L(a) = a \cdot P_1 + (1-a) \cdot P_2$$

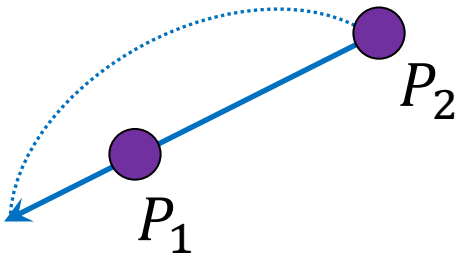
6.3.1 두 점의 결합

Heesung Oh
Copyright All rights reserved

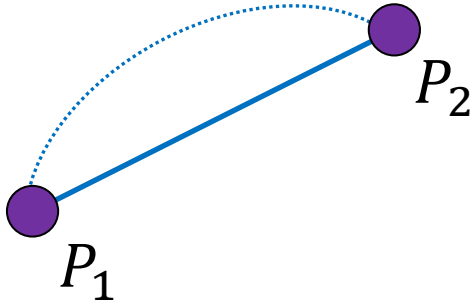
- **직선**
  - $-\infty < a < \infty$



- **반직선**
  - $0 \leq a < \infty$



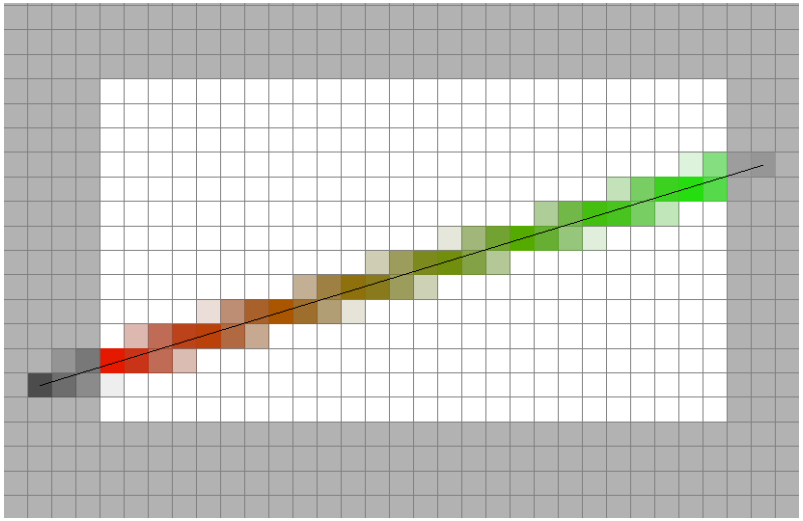
- **선분**
  - $0 \leq a \leq 1$



6.4.1 벡터를 모니터의 점으로 표현

Heesung Oh
Copyright All rights reserved

- 라인 레스터 화(Line Rasterization)
  - 두 점을 연결하는 선을 픽셀 단위로 근사하여 표현하는 과정



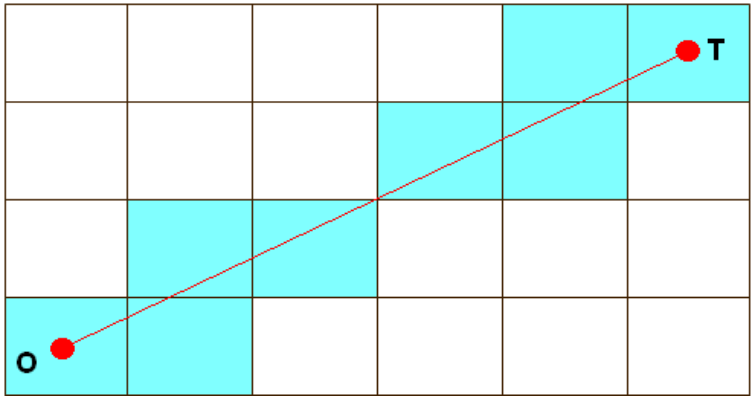
  - 왼손 좌표계 화면 + 중점 사용
    - 좌상단 (0,0) → 우하단 (Screen W, Screen H)

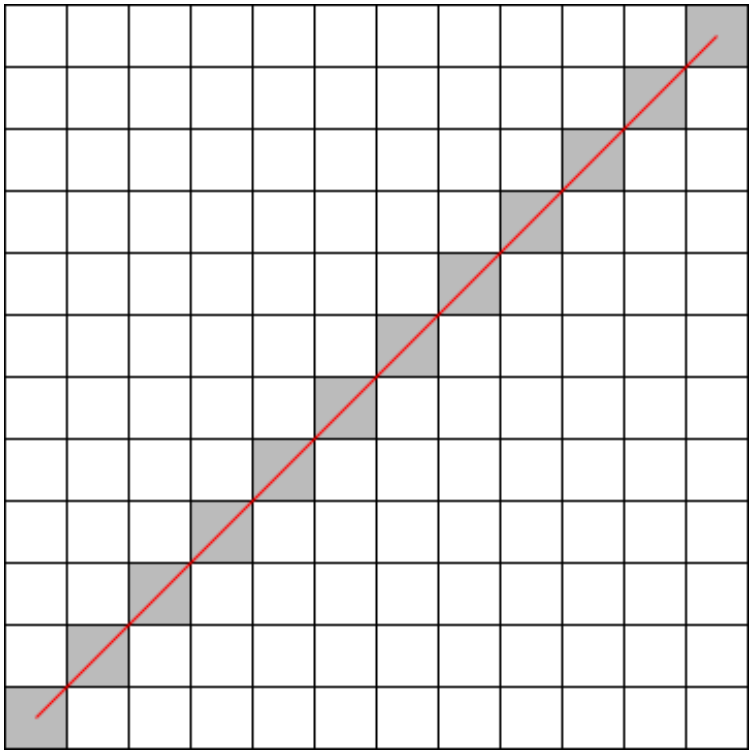
```
ScreenPoint(InPos.X + InScreenSize.X * 0.5f
          , -InPos.Y + InScreenSize.Y * 0.5f);
```

6.4.2 선 그리기 알고리즘

Heesung Oh
Copyright All rights reserved

- 브레젠험 알고리즘 (Bresenham)
  - 정수 연산만 사용
  - 오차 누적으로 픽셀 선택
  - 빠름 (GPU 이전 시대 핵심)
  - 가장 널리 사용됨
- 동작 원리
  - 현재 픽셀에서 다음 픽셀 결정
  - 두 후보 중 더 가까운 픽셀 선택
  - 오차(error) 누적하여 판단
- 처리 과정
  - 시작점 → 끝점 설정
  - 기울기 계산
  - 한 축 기준으로 반복
  - 오차 값으로 픽셀 선택

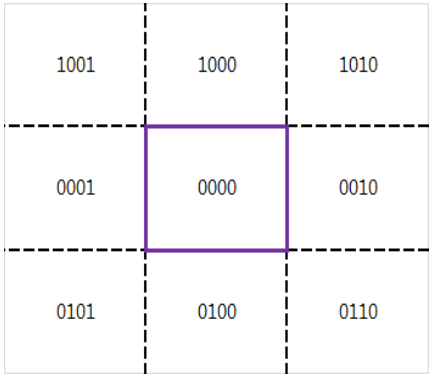




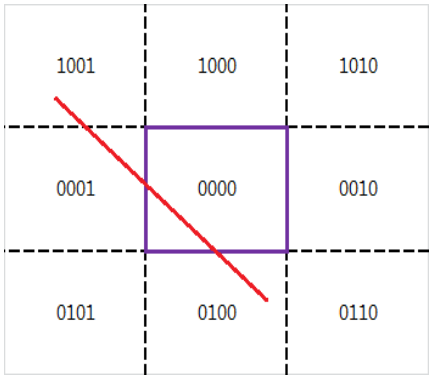
6.4.3 라인 클리핑

Heesung Oh
Copyright All rights reserved

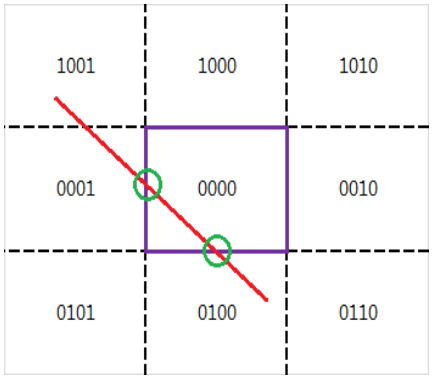
- 코헨-서덜렌드 라인 클리핑
  - 직선 vs 사각형 영역 클리핑
  - 빠른 판정



- Outcode
  - 점 위치: 4비트(0000)
    - 10XX 상 01XX 하
    - XX01 좌 XX10: 우



- 판정
  - OR: 하나라도 밖에 있는가?
  - AND: 둘 다 밖에 있는가?
  - 점 P1 → code1, P2 → code2
    - code1 | (OR) code2 == 0
      → 그대로 사용 (Trivial Accept)
    - code1 & (AND) code2 ≠ 0
      → 절대 내부로 못 들어옴. 버림 (Trivial Reject)



6.정리

Heesung Oh
Copyright All rights reserved

- 아핀 공간
  - 점 과 벡터의 연산으로 구성된 공간
  - 점 = 점 + 벡터

- 아핀 공간 특성
  - 점 – 점 = 벡터
  - 점 +벡터 = 점
  - 벡터 + 벡터 = 벡터
  - 점 + 점 ≠ 점

- 아핀 공간의 선형 결합

$$P = \frac{\sum c_i \cdot P_i}{\sum c_i}$$